TP10 — Première bataille en console

Algorithme et modélisation avancée | Java | Début du projet évolutif

Objectif général : quitter les exercices isolés et assembler un premier noyau de RPG tactique en Java. À la fin du TP, une phase alliée puis une phase ennemie s'exécutent en console, avec actions explicites, PV, états et vérification des fins de phase. Aucune grille ni interface graphique à ce stade.

Règles du travail

Les 10 exercices se suivent : tu enrichis les mêmes fichiers, sans créer une nouvelle classe Exercice1, Exercice2, etc. Tu écris toi-même les implémentations ; les signatures proposées sont des repères, pas du code à recopier. Nous corrigerons chaque étape avant de passer à la suivante.

Réutilise tes classes et connaissances des TP précédents : Unite, Epeiste, Cavalier, Mage, Clerc, Soigneur, EtatUnite, ArrayList, HashMap, ainsi que subirDegats, soignerCible et attaqueRapide. Ne recopie pas tes classes Exercice4 ou Exercice9 : déplace ou réécris la logique utile dans des classes du projet.

Scénario de référence (identique pendant tout le TP)

Équipe alliée, dans cet ordre : Chrom (Épéiste, 24/24 PV, force 8), Frederick (Cavalier, 30/30 PV, défense 7), Robin (Mage, 22/22 PV, magie 9), Lissa (Clerc, 18/18 PV). Équipe ennemie : Vaike (Épéiste, 25/25 PV, force 6). Tous commencent DISPONIBLE. Chaque unité agit au maximum une fois par phase.

Règle de combat du TP10 : une attaque rapide inflige directement la force de l'Épéiste (comme au TP6) ; aucune défense, portée, distance, mouvement, riposte ou IA. Les dégâts reçus par Chrom avant le soin sont un événement scénarisé, et non une action ennemie pendant la phase alliée.

Exercice 1 — Un point d'entrée unique

Travail demandé. Créer un petit projet Java contenant au minimum Main (avec main) et tes classes de personnages réutilisées. Instancier les cinq unités du scénario. Afficher leur nom, leur type et leurs PV.

Contrainte. Ne pas instancier Unite directement. Une seule méthode main sert de point d'entrée.

Validation. Les cinq unités possèdent exactement les statistiques indiquées.

Exercice 2 — Former les deux équipes

Travail demandé. Créer une ArrayList<Unite> pour les quatre alliés et une autre pour l'ennemi. Écrire une méthode qui affiche une équipe à partir de la liste reçue, puis l'utiliser pour les deux camps.

Contrainte. La méthode ne doit supposer ni quatre alliés ni un ennemi ; éviter de coder les indices de façon fixe.

Validation. L'ordre d'affichage des alliés est Chrom, Frederick, Robin, Lissa ; Vaike figure seulement parmi les ennemis.

Exercice 3 — État d'activation des unités

Travail demandé. Créer une HashMap<Unite, EtatUnite> et initialiser à DISPONIBLE les unités des deux équipes. Ajouter une méthode permettant de récupérer l'état d'une unité et l'afficher pour les cinq personnages.

Contrainte. Associer les états aux objets et non à leurs indices. Prévoir l'ajout futur d'unités sans modifier la logique.

Validation. Toutes les unités affichent DISPONIBLE au début.

Exercice 4 — Autoriser une action

Travail demandé. Écrire une méthode de validation, par exemple peutAgir(Unite unite), qui n'autorise une action que si l'unité a des PV strictement positifs et l'état DISPONIBLE. Tester une unité disponible, une unité TERMINEE et une unité à 0 PV.

Contrainte. La méthode doit seulement renvoyer un booléen, sans afficher ni modifier l'unité. Pour les tests, rétablir les états nécessaires ensuite.

Validation. Les résultats sont respectivement true, false, false.

Exercice 5 — Première action : attaquer

Travail demandé. Pendant la phase alliée, faire attaquer Chrom sur Vaike avec attaqueRapide(), uniquement si Chrom peut agir. Une fois l'attaque réalisée, passer Chrom à TERMINEE. Afficher les PV de Vaike.

Contrainte. Ne pas modifier directement les PV ni ignorer la validation de l'exercice 4. Une tentative de seconde action de Chrom ne doit rien appliquer.

Validation. Vaike après attaque : 17 / 25 PV ; Chrom est TERMINEE.

Exercice 6 — Action d'attente

Travail demandé. Créer une opération explicite d'attente, par exemple attendre(Unite unite). Faire attendre Frederick puis Robin pendant la même phase alliée.

Contrainte. Attendre ne modifie pas les PV ; l'opération ne doit fonctionner que sur une unité autorisée à agir. Ne pas construire de moteur automatique à partir de l'enum Action.

Validation. Frederick et Robin deviennent TERMINEE et leurs PV restent inchangés.

Exercice 7 — Dégâts reçus et soin

Travail demandé. Avant l'activation de Lissa, faire subir 6 dégâts à Chrom via subirDegats(). Faire ensuite soigner Chrom de 5 PV par Lissa via l'interface Soigneur, puis marquer Lissa TERMINEE. Afficher les PV de Chrom.

Contrainte. Lissa ne soigne que si elle peut agir ; réutiliser le plafonnement des PV de Unite.soigner(). Les 6 dégâts sont imposés par le scénario, sans attaque supplémentaire de Vaike.

Validation. Chrom après dégâts et soin : 23 / 24 PV.

Exercice 8 — Détecter la fin de la phase alliée

Travail demandé. Créer une vérification globale qui reçoit une équipe et les états, et renvoie vrai seulement si toutes les unités de cette équipe sont TERMINEE. L'utiliser à la fin des quatre actions alliées.

Contrainte. Ne pas supposer qu'une équipe contient toujours quatre unités ; ne pas tenir compte des états ennemis pour décider si la phase alliée est terminée.

Validation. Phase alliée terminée : true. Avant l'action de Lissa, la même vérification doit renvoyer false.

Exercice 9 — Passer à la phase ennemie

Travail demandé. Après la fin de la phase alliée, simuler la phase ennemie : Vaike, s'il peut agir, utilise attaqueRapide() sur Chrom, puis devient TERMINEE. Vérifier la fin de la phase ennemie.

Contrainte. Vaike reste à 17/25 PV. Il s’agit d’une action scénarisée, pas d’une IA. Ne réinitialise pas les états alliés avant la fin de la phase ennemie.

Validation. Chrom après phase ennemie : 17 / 24 PV et Phase ennemie terminée : true.

Exercice 10 — Préparer le tour suivant

Travail demandé. Créer une méthode réinitialisant à DISPONIBLE l’état des unités vivantes de l’équipe reçue. Après les deux phases terminées, réinitialiser les deux équipes et afficher leurs états. Organiser les appels pour que le déroulement complet reste lisible dans main (ou dans une classe Partie facultative).

Contrainte. Ne pas réanimer d’unité à 0 PV ; ne pas effacer les PV ni recréer les personnages. Le deuxième tour n’a pas besoin d’être simulé.

Validation. Chrom conserve 17/24 PV, Vaike 17/25 PV ; tous les personnages vivants sont de nouveau DISPONIBLE.

Sortie minimale attendue en fin de TP

```
Vaike après attaque : 17 / 25 PV
Chrom après dégâts et soin : 23 / 24 PV
Phase alliée terminée : true
Chrom après phase ennemie : 17 / 24 PV
Phase ennemie terminée : true
Début du tour suivant : unités vivantes DISPONIBLE
```

Critères de validation

Chaque exercice est noté sur 10 selon ses consignes explicites. Priorité : comportement correct, absence de modification directe des attributs privés, gestion des états, réutilisation des méthodes et possibilité de faire évoluer les effectifs. Un affichage supplémentaire pour vérifier ton travail est autorisé.

Hors périmètre — pour les TP suivants

Pas encore de grille, déplacement, terrain, portée, armes, inventaire, défense systématique, adversaire intelligent, menus, JavaFX ou réseau. L’objectif du TP10 est un premier socle de partie cohérent, pas un jeu complet.

Étape suivante après réception du sujet : créer ensemble le dossier du projet et le dépôt GitHub, puis commencer l’exercice 1. Nous choisirons à ce moment-là l’organisation précise des fichiers et les premiers commits.